Customer Intelligence & Recommendation Engine: Project Report

Prepared from the full build log of the project. Dataset: H&M Personalized Fashion Recommendations (Kaggle). Repository: github.com/swathimol/customer-intelligence-api

1. Executive summary

This project builds an end-to-end customer analytics system on a sample of the H&M transaction data. It answers three business questions with one dataset: who is likely to buy again in the next 90 days, what kinds of customers exist, and what each customer should be shown next. It also covers the engineering around the models: explainability, experiment tracking, drift monitoring, a REST API, containerisation and automated tests.

Key results:

- A leak-free 90-day repurchase classifier reached **0.816 test ROC-AUC** on an out-of-time 2020 snapshot, against 0.761 for a recency-only benchmark.
- All four model families (Logistic Regression, Random Forest, XGBoost, LightGBM) tied on ranking quality. The features, not the algorithm, carry the signal.
- The main finding is a **calibration drift**: the actual repurchase rate fell from 0.476 to 0.362 while the model kept predicting about 0.448. Input and score monitoring both looked stable, so they would have missed it.
- K-Means on log-scaled RFM produced four interpretable segments with repurchase rates from 0.26 to 0.78.
- A recency-aware hybrid recommender beat a 14-day popularity baseline by **135% MAP@12** on a held-out 2020 window, after a plain cosine recommender had lost to the baseline.
- The FastAPI service reproduces notebook outputs exactly on Kaggle, on Windows (Python 3.11) and inside a Python 3.13 Docker image, and is covered by 7 automated tests.

2. Objectives

1. Predict whether a customer will purchase again within 90 days.
2. Segment customers into actionable groups.
3. Recommend articles to individual customers.

4. Explain model behaviour and check it for leakage.
5. Track experiments and monitor drift over time.
6. Serve everything through a tested, containerised API.

3. Data

3.1 Source and sampling

The full transactions file holds about 31 million rows, which is too large for a laptop. Customers (not rows) were sampled so that each kept their full purchase history. 150,000 customers were drawn with a fixed seed, and 148,939 of them had at least one transaction.

Item	Value
Sampled transaction rows	3,477,131
Customers with transactions	148,939
Distinct articles	87,289
Date range	2018-09-20 to 2020-09-22
Customer-day "orders"	994,343
Nulls in transactions	0

3.2 Cleaning

- Duplicate rows were collapsed into a quantity column.
- Long hash customer IDs were replaced with integer indices, with a saved mapping.
- Missing age was filled with the median. Missing club and news-frequency fields were set to a NONE category, and missing flags to 0.
- Postal code was dropped. Only articles that were actually sold were kept.
- Because the data has no invoice number, an "order" is defined as all items a customer bought on the same date.

3.3 Data caveats

- Prices are anonymised, so monetary values are relative, not real currency.

- There is no cancellation table, so no return-rate feature exists.
- The data starts in September 2018, so customer tenure is left-censored.

4. Methodology

4.1 Leak-free snapshot design

For each cutoff date, features use only transactions on or before the cutoff, and the label asks whether the customer bought in the following 90 days. Training label windows end before the validation cutoff, so nothing overlaps.

Role	Cutoff	Customers	Repurchase rate
Train 1	2019-03-31	83,481	0.511
Train 2	2019-06-30	100,076	0.448
Validation	2019-09-30	110,912	0.415
Test (out of time)	2020-03-31	130,565	0.362
Drift only	2020-06-22	140,710	0.352

The combined training set has 183,557 rows with a base rate of 0.476.

4.2 Features (25)

- **RFM and rhythm:** recency, frequency, monetary, average basket value and size, tenure, average days between orders, online share.
- **Recent activity:** orders and spend in the last 30 and 90 days.
- **Product behaviour:** distinct products, distinct product types, average price, and the share of purchases in each department group.
- **Customer attributes:** age, FN, Active, club member status, fashion news frequency.

4.3 Performance note

The first feature build was slow. Pre-aggregating once to customer-day level and counting distinct products through first-purchase dates cut each snapshot to about one second, with identical output.

5. Classification results

Models were trained on the two training snapshots, early-stopped on validation, and scored once on the test snapshot. Because the classes are reasonably balanced, ROC-AUC and PR-AUC were reported instead of accuracy.

Model	Val AUC	Test AUC	Test PR-AUC	Test logloss
Recency-only benchmark	0.7607	0.7608	0.6037	n/a
Logistic Regression	0.8273	0.8165	0.7295	0.5590
Random Forest	0.8274	0.8166	0.7334	0.5123
XGBoost	0.8283	0.8163	0.7311	0.5197
LightGBM	0.8280	0.8164	0.7317	0.5180

Observations:

- No sign of leakage: AUC sits near 0.82, well below the 0.95 level that would signal a problem.
- Every model beats the recency benchmark by about 0.06 AUC and 0.12 to 0.13 PR-AUC.
- Test AUC is only about 0.01 below validation, so ranking quality survived the 2020 shift.
- The four models are statistically indistinguishable on AUC. The automatic "best by validation PR-AUC" label (Logistic Regression) was a tie-break artifact and should not be claimed as a win.
- Logistic Regression has the worst test logloss, so its probabilities degrade more under shift.

Deployed model: LightGBM. It was chosen on a rule set in advance: validation logloss, then serving cost. It is far lighter than a 300-tree forest, handles missing values natively and works with fast SHAP explanations.

6. Calibration

Ranking held up, but probabilities did not. On the test snapshot the mean predicted probability was 0.448 against an actual rate of 0.362, and every calibration bin sat below the diagonal.

Predicted	Actual
0.08	0.04
0.25	0.17
0.45	0.33
0.65	0.51
0.85	0.76
0.93	0.92

This is prior shift: the base rate fell and the model kept predicting as if the old rate held. Platt scaling and isotonic regression were fitted on validation and evaluated on test.

Version	Mean predicted	AUC	Logloss	Brier
Raw LightGBM	0.448	0.8164	0.5180	0.1719
Platt	0.396	0.8164	0.5004	0.1651
Isotonic	0.396	0.8163	0.4997	0.1649

Platt was shipped because the difference from isotonic is in the third decimal and it is smoother and less prone to overfitting. The remaining gap (0.396 vs 0.362) exists because the calibrator was fitted on a validation base rate of 0.415. No static calibrator can remove it, which is why monitoring and retraining triggers are needed.

7. Explainability and leakage check

7.1 SHAP

SHAP values were computed on a 5,000-customer test sample using the LightGBM TreeExplainer.

Rank	Feature	Mean abs SHAP
1	frequency	0.740
2	recency	0.376

Rank	Feature	Mean abs SHAP
3	avg_days_between	0.249
4	tenure	0.105
5	club_member_status	0.094
6	orders_last_90d	0.070
7	share_baby_children	0.068
8	Active	0.055
9	n_types	0.045
10	online_share	0.042

The top four are behavioural: how often, how recently, how regularly and for how long a customer buys. Direction checks matched intuition: the correlation between feature value and SHAP is +0.81 for frequency and -0.82 for recency. Average days between orders had a weak +0.14, which reflects a non-linear effect, so no direction is claimed for it.

7.2 Leakage ablation

club_member_status and Active ranked 5th and 8th, which was unexpected. They come from the end-of-data customer table rather than a value as of each cutoff, so they could carry mild future information. Retraining without the four static attributes gave:

Model	Features	Test AUC	Test logloss
With static attributes	25	0.8164	0.5180
Without	21	0.8142	0.5187

The drop of 0.002 AUC is small, so no material leakage was detected and the original model was kept. The conservative claim is "no material leakage detected", not "no leakage".

8. Segmentation

K-Means was fitted on validation-snapshot RFM features (recency, frequency, monetary, average basket value, tenure) after log transformation and scaling. Silhouette peaked at k=4.

k	Silhouette
3	0.345
4	0.359
5	0.341
6	0.321
7	0.313

A silhouette of 0.36 is moderate. RFM data is a continuum, so the segments are useful and interpretable rather than sharply separated.

Segment	Profile (medians)	Share	Repurchase rate
Loyal core	20 days since last order, 9 orders, long tenure	28%	0.781
High-value lapsing	126 days, 2 orders, largest baskets	8%	0.328
Lapsed low-value	183 days, 2 orders, small baskets	51%	0.258
New customers	1 order, about 2 months tenure	12%	0.293

From validation to test, the new-customer share fell by 4.7 points and the lapsed low-value share rose by 4.0 points. Part of that is mechanical, because recency and tenure grow as data ages, so it is described as consistent with fewer new customers rather than attributed to COVID. High-value lapsing customers are the natural retention target: they spend a lot per order but are drifting away.

9. Recommender

9.1 First attempt failed

A plain item-item cosine recommender over all history scored MAP@12 of 0.0046 against 0.0068 for an all-time popularity baseline. It lost. The diagnosis: fashion is recency-driven, so long-run popularity and long-run co-purchase are weak signals.

9.2 Fix

- Candidate set limited to the top 5,000 items sold in the last 30 days.
- Customer-item matrix built from the last 180 days with a 30-day half-life time decay.
- Item-item cosine similarity, blended with a 14-day popularity vector: score = normalised cosine + alpha x popularity.
- The honest baseline became 14-day popularity.

9.3 Tuning and held-out confirmation

Alpha was tuned on the 2019-09-30 cutoff and then fixed. It was scored once on 2020-03-31 without retuning.

Cutoff	Method	MAP@12	Hit-rate@12
2019-09-30 (tuned)	14-day popularity	0.0053	0.0774
2019-09-30 (tuned)	Hybrid, alpha 1.0	0.0071	0.0930
2020-03-31 (held out)	14-day popularity	0.0023	0.0418
2020-03-31 (held out)	Hybrid, alpha 1.0	0.0054	0.0690

Alpha 1.0 was a true optimum on the wider grid: 1.5 gave 0.0065 and 5.0 gave 0.0056. The held-out lift is **+135% MAP@12 and +65% hit-rate@12**. The popularity baseline collapsed in 2020 while the hybrid fell much less, which suggests personalisation was more valuable when behaviour changed.

Caveats: absolute scores are small, which is normal for fashion with 5,000 candidates. Evaluation covers customers who bought in the test window, so it measures ranking among buyers, not who will buy. Customers with no recent history (about 55%) fall back to the popular list.

10. Drift monitoring

PSI and KS statistics compare each feature in the training period with the later snapshots, with bins fixed on training data. PSI below 0.1 is stable, 0.1 to 0.25 moderate and above 0.25 major.

Feature	PSI val	PSI test	PSI late	Flag (test)
tenure	1.283	2.359	2.493	MAJOR
recency	0.242	0.560	0.610	MAJOR
avg_days_between	0.070	0.261	0.366	MAJOR
orders_last_90d	0.059	0.212	0.197	moderate
spend_last_90d	0.061	0.209	0.180	moderate

On the test snapshot, 17 features were stable, 5 moderate and 3 major. Most of the major flags are mechanical: the data starts in September 2018, so histories lengthen over time. The data cannot separate COVID from this effect.

10.1 Score and label drift: the headline

Snapshot	Mean score	Actual rate	Gap	Score PSI
Train	0.477	0.476	0.001	n/a
Validation	0.467	0.415	0.052	0.031
Test	0.448	0.362	0.085	0.062
Drift late	0.447	0.352	0.095	0.071

The score distribution barely moved (PSI below 0.1) while the actual rate fell by about 12 points. A monitor on inputs and scores alone would have reported stability while the model over-predicted by up to 9.5 points. Monitoring therefore needs predicted-versus-actual rates once labels arrive.

10.2 Operating rules

Trigger	Action
Feature PSI above 0.25 for two consecutive snapshots	Investigate whether the cause is mechanical
Predicted minus actual rate above 0.05	Recalibrate with Platt on the latest labelled snapshot
Gap above 0.10, or AUC drop above 0.03	Retrain on recent snapshots
Tenure	Exclude from alerts, since it drifts by construction

The current largest gap (0.095) sits between the recalibration and retraining thresholds. The standalone script src/drift.py reproduces the notebook numbers and prints the matching action.

11. Experiment tracking

Runs were logged to MLflow using a SQLite backend after the default file store was blocked in the newer MLflow version. Parameters and metrics were logged for all models. XGBoost and LightGBM logged model artifacts. The Logistic Regression and Random Forest artifacts failed on a serialisation safety check, so those two runs hold metrics and parameters only, while their model files are saved separately.

12. Serving and deployment

12.1 API

Endpoint	Purpose
GET /health	Status, feature count, recommender cutoff
GET /features	The 25 expected feature names
POST /predict	Raw score and Platt-calibrated 90-day probability
POST /segment	RFM segment id and name

Endpoint	Purpose
GET /recommend/{cust_idx}	Top-N articles, hybrid or popular fallback

Models load once at startup. Inputs are validated: missing or unknown features and negative RFM values return HTTP 422. Null is accepted for average days between orders.

12.2 Parity verification

A reference customer was scored in the notebook (raw 0.8940, probability 0.8740, segment Loyal core). The API reproduced it in three environments:

Environment	Python	Result
Kaggle notebook	3.13	0.8940 / 0.8740
Windows virtual environment	3.11	0.894 / 0.874
Docker container (GitHub Codespaces)	3.13	0.894 / 0.874

Library versions were pinned to those used to create the pickles, which is the main safeguard against silent differences.

12.3 Tests

Seven pytest tests passed: health, prediction parity with the notebook, missing-feature rejection, segment parity, negative-value rejection, hybrid recommendation for a known user and popular fallback for an unknown user.

13. Limitations

- Only 150,000 sampled customers were used, so absolute numbers may differ on the full data.
- Prices are anonymised, so monetary values are relative.
- Age and club status come from the end-of-data customer table. The ablation shows the effect is small, but a slight leak cannot be fully ruled out.
- Tenure drift is mostly mechanical and cannot be separated from COVID effects in this data.

- The repurchase rate falls partly because early snapshots contain only recently active customers, so the population changes between snapshots.
- The Platt calibrator was fitted on 2019-09 data and still over-predicts in 2020.
- The recommender is evaluated only on customers who later bought, and offline metrics may not match live performance.
- The feature pipeline uses pandas, not SQL. A resume claim of SQL skills needs a SQL component (for example a SQLite query that rebuilds the RFM features) to be accurate.
- Two MLflow runs lack model artifacts.

14. Recommendations and next steps

1. **Add a labelled-data monitor:** compute predicted-versus-actual rate each month and apply the operating rules above.
2. **Retrain on recent snapshots** (train plus validation) and compare calibration with Platt scaling. Recalibration is the cheaper first response.
3. **Add a SQL step:** rebuild the RFM features with a SQLite query and check they match the pandas output.
4. **Fix the two MLflow artifacts** using cloudpickle serialisation if registered models are to be claimed.
5. **Add charts to the repository:** SHAP bar and beeswarm plots and the PSI chart in a docs folder.
6. **Add input range checks and batch scoring** to the API.
7. **Restrict the population** to customers active in the last 365 days if a more stable repurchase rate is wanted.

15. Reproducibility

- Kaggle notebook steps, in order: clean and sample, build features, train and log models, calibrate, segment, recommender, SHAP and ablation, drift report, bundle API artifacts.
- Local: create a virtual environment, install the pinned requirements, run uvicorn on app.main:app, then pytest.
- Docker: build the image from the Dockerfile (python:3.13-slim with the OpenMP runtime) and run it on port 8000.

- Drift: run src/drift.py with the five feature Parquet files in a data folder.

16. Resume summary

- Built a leak-free 90-day repurchase pipeline on 3.5M H&M transactions (150k customers) with time-based train, validation and out-of-time test snapshots. LightGBM, XGBoost and Random Forest reached 0.82 test AUC against a 0.76 recency benchmark.
- Segmented customers with K-Means on log-scaled RFM (4 segments, repurchase rates 0.26 to 0.78) and built a time-decayed item-similarity recommender that beat a popularity baseline by 135% MAP@12 on a held-out 2020 window.
- Used SHAP for explainability, tested for leakage by ablation (0.002 AUC cost), tracked experiments in MLflow, and built PSI/KS drift monitoring. Found predictions over-stated the repurchase rate by 9.5 points while score PSI stayed below 0.1, and responded with Platt recalibration and retrain triggers.
- Shipped a FastAPI service in Docker with a pytest suite that checks it reproduces notebook outputs exactly.